



United International University
Department of CSE

Course Code: CSE 426/CSE 4326

Course Name: Microprocessor, Microcontroller and Interfacing Lab

Experiment no: 4

Experiment Name: : Image and Video Processing with Raspberry Pi by
interfacing pi camera

Submitted by:

Name: Mithila Farjana

Student ID: 011 221 575

Section: H

Date of Performance: Aug 19, 2026

Date of Submission: Sep 01, 2026

Objective:

- To learn how to capture and stream live video from the Raspberry Pi Camera Module, and from a mobile phone camera used as an external webcam, using Python and OpenCV.
- To understand how a customized, locally-trained machine learning model can be converted and deployed on a resource-constrained microcontroller (ESP32 with ESP32-CAM) using TinyML techniques.
- To understand how a live video feed is internally processed, frame by frame, by a TensorFlow Lite object detection pipeline running on the Raspberry Pi.

Components:

Raspberry Pi 4 (with Raspberry Pi OS installed), Raspberry Pi Camera Module (CSI interface), micro SD card, monitor/keyboard/mouse for initial setup, mobile phone with the “IP Webcam” application (used as an external network camera), a Wi-Fi network connecting the Pi and the phone, an ESP32-CAM (AI-Thinker) module, a USB-to-serial (FTDI) programmer for flashing the ESP32-CAM, and jumper wires.

Theory:

The Raspberry Pi Camera Module connects to the Pi's CSI (Camera Serial Interface) port and can be accessed programmatically in Python through the picamera library, which exposes the camera as a continuous stream of frames. These frames can be captured into NumPy arrays using `PiRGBArray` and passed to OpenCV (`cv2`), an open-source computer vision library, for real-time display, processing, and analysis. A live video window can also be fed from a device that is not physically wired to the Pi at all — for example, a mobile phone can run an “IP Webcam” style application that publishes its camera feed as an MJPEG stream over Wi-Fi at a URL such as `http://<phone-ip>:8080/video`. OpenCV's `VideoCapture()` function can open this network URL exactly as it would open a local camera index, which is what allows a phone to act as a wireless webcam for the Pi or a PC.

On the machine-learning side, TensorFlow Lite is a lightweight version of TensorFlow designed to run trained models efficiently on mobile and embedded devices. A model that has been trained and validated on a full-size computer is first converted to the compact `.tflite` format, and can be further shrunk using post-training quantization (typically to 8-bit integers), trading a small amount of accuracy for a large reduction in size and inference time. For very small microcontrollers such as the ESP32, this idea is taken a step further under the umbrella of TinyML — running quantized neural networks directly on microcontroller-class hardware with only a few hundred kilobytes of RAM, using a stripped-down runtime called TensorFlow Lite Micro. Since microcontrollers do not have a general-purpose filesystem the way a PC does, the `.tflite` model is not loaded from disk at runtime; instead, it is converted into a C byte array (using a tool such as `xxd`) and compiled directly into the firmware.

For the object detection example in this experiment, the Raspberry Pi runs a pre-trained SSDLite-MobileNet-v2 model, trained on the MSCOCO dataset to recognize 80 common object classes. This is a Single Shot Detector (SSD) architecture built on a lightweight MobileNet backbone, which makes it fast enough to run in real time on a Raspberry Pi's CPU. For every video frame, the model outputs a set of candidate bounding boxes, each with a predicted class and a confidence score, which the surrounding Python script then filters and draws on top of the

live video feed.

Task 1 — Live Video Streaming (RPi Camera + Mobile Camera as Webcam)

Both scripts below use OpenCV ([cv2](#)), since it gives a uniform way to grab frames, display them, and check for a key press — the legacy [picamera](#) library used in Experiment 4 doesn't have a built-in "press q to stop" preview loop the way OpenCV does.

Install OpenCV first (inside your virtual environment or system-wide):

```
pip3 install opencv-python
```

a) Stream live video from the Raspberry Pi Camera

```
import cv2

# 0 usually maps to the Pi camera (via the legacy/libcamera V4L2 compatibility layer)
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Error: Could not access the Raspberry Pi camera.")
    exit()

while True:
    ret, frame = cap.read()
    if not ret:
        print("Failed to grab frame.")
        break

    cv2.imshow("RPi Camera - Live Stream", frame)

    # Wait 1 ms between frames, quit when 'q' is pressed
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

b) Stream live video using your mobile phone as a webcam

The simplest method is installing an app like **IP Webcam** (Android) or **DroidCam**, starting the server on the phone, and connecting to its stream URL over the same Wi-Fi network:

```
import cv2
```

Replace with the IP address and port shown in your IP Webcam app

```
url = "http://192.168.1.5:8080/video"
```

```
cap = cv2.VideoCapture(url)
```

```
if not cap.isOpened():
```

```
    print("Error: Could not connect to mobile camera stream.")
```

```
    exit()
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        print("Failed to grab frame.")
```

```
        break
```

```
cv2.imshow("Mobile Camera - Live Stream", frame)
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

Task 2: Running a Customized Machine Learning Security Model on ESP32 Using an ESP Camera

Introduction

Suppose a software company needs a low-cost security system that can distinguish between **authorized employees** and **people outside the organization**. A machine-learning model can first be trained on a local computer using a prepared dataset. However, the trained desktop model cannot simply be copied directly to an ESP32 because the ESP32 has limited processing power and memory. Therefore, the model must be converted and optimized for embedded inference using **TensorFlow Lite for Microcontrollers (TFLite Micro)** or another TinyML-compatible framework.

Espressif provides `esp-tflite-micro`, which integrates TensorFlow Lite Micro with ESP-IDF and provides examples, including ML inference examples.

Step 1: Define the classification problem

The first step is deciding exactly what the model should recognize.

For a simple version of the security system, the classes could be:

- `Employee`
- `Outsider`

Alternatively, if the company wants to identify individual employees, the classes may be:

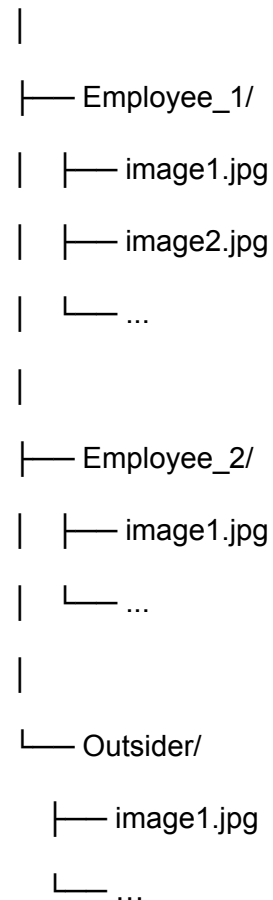
- `Employee_1`
- `Employee_2`
- `Employee_3`
- `...`
- `Unknown/Outsider`

For an ESP32, a simpler classification problem is generally easier to deploy because embedded devices have limited memory and processing resources.

Step 2: Prepare the dataset

The dataset should contain many images for every class.

For example:dataset/



The dataset should contain variation in:

- Lighting conditions
- Camera angle
- Distance from camera
- Facial expression
- Background
- Indoor and outdoor environments

The dataset is then divided into:

- **Training dataset** – used to teach the model.
- **Validation dataset** – used during model development and tuning.
- **Test dataset** – used to evaluate the final model.

The important point is that the model should be tested on images that were not simply reused from its training set.

Step 3: Train the ML model on a local computer

A computer with sufficient processing capability can be used to train a lightweight image-classification model.

For example, TensorFlow/Keras can be used to create a CNN:

```
model = tf.keras.Sequential([  
    tf.keras.layers.Rescaling(1./255, input_shape=(96, 96, 3)),  
  
    tf.keras.layers.Conv2D(8, 3, activation='relu'),  
    tf.keras.layers.MaxPooling2D(),  
  
    tf.keras.layers.Conv2D(16, 3, activation='relu'),  
    tf.keras.layers.MaxPooling2D(),  
  
    tf.keras.layers.Flatten(),  
    tf.keras.layers.Dense(32, activation='relu'),  
    tf.keras.layers.Dense(number_of_classes, activation='softmax')  
])
```

The model is trained using the prepared dataset.

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)  
  
model.fit(train_dataset, validation_data=validation_dataset, epochs=10)
```

The model selected for deployment should be small enough for the target hardware. A model with very high desktop accuracy but excessive memory requirements may not be practical for an ESP32.

Step 4: Convert the trained model to TensorFlow Lite

After training, the Keras/TensorFlow model is converted into a `.tflite` model.

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
```

```
tflite_model = converter.convert()
```

```
with open("security_model.tflite", "wb") as f:
```

```
    f.write(tflite_model)
```

Step 5: Quantize the model for TinyML

This is one of the most important steps.

A normal floating-point model can be too large or computationally expensive for an embedded system. **Quantization** reduces the numerical precision used by the model, commonly converting floating-point operations to smaller integer representations such as `int8`.

A representative dataset can be supplied during full integer quantization so that the converter can determine appropriate quantization ranges.

```
def representative_dataset():
```

```
    for image in calibration_dataset:
```

```
        yield [image]
```

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
```

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
```

```
converter.representative_dataset = representative_dataset
```

```
converter.target_spec.supported_ops = [  
    tf.lite.OpsSet.TFLITE_BUILTINS_INT8  
]
```

```
converter.inference_input_type = tf.int8  
converter.inference_output_type = tf.int8
```

```
tflite_model = converter.convert()
```

```
with open("security_model_int8.tflite", "wb") as f:
```

```
    f.write(tflite_model)
```

Step 6: Set up the ESP32 development environment

A practical approach is to use **ESP-IDF**, Espressif's development framework.

The current [esp-tflite-micro](#) project documents installation through ESP-IDF and provides TensorFlow Lite Micro examples.

After ESP-IDF is installed, an ESP-IDF project can include the TFLite Micro component with:

```
idf.py add-dependency "esp-tflite-micro"
```

The official component repository also documents creating example projects with:

```
idf.py create-project-from-example "esp-tflite-micro:<example_name>"
```

and then building and flashing them using [idf.py](#).

A useful starting point is to first build and run a known TensorFlow Lite Micro example before integrating the custom model.

Step 7: Configure the ESP camera

The ESP camera must be initialized before inference.

The camera configuration includes information such as:

- Camera model/pin configuration
- Frame size
- Pixel format
- Frame buffer
- Camera clock and synchronization pins

Espressif's TFLite Micro example configuration includes support for several camera configurations, including the **AI-Thinker ESP32-CAM** and custom camera pinouts.

Step 8: Convert the `.tflite` model into a C/C++ array

The ESP32 firmware needs access to the model. One common embedded approach is to convert the `.tflite` binary into a C array.

Step 9: Create the TensorFlow Lite Micro interpreter

The ESP32 application must:

1. Load the model.
2. Register the TensorFlow Lite operations required by that model.
3. Create a `MicroInterpreter`.
4. Allocate memory for tensors.
5. Obtain the input and output tensors.

The Espressif TFLite Micro example follows this pattern: it creates a `MicroInterpreter`, allocates tensors, and obtains the model input before performing inference.

Conceptually:

```
const tflite::Model* model =
```

```
    tflite::GetModel(security_model);
```

```
tflite::MicroInterpreter interpreter(
```

```
    model,
```

```
    resolver,
```

```
    tensor_arena,
```

```
    tensor_arena_size
```

```
);
```

```
interpreter.AllocateTensors();
```

```
TfLiteTensor* input = interpreter.input(0);
```

Step 10: Capture and preprocess an image

The ESP camera captures a frame.

Step 11: Run ML inference

After filling the input tensor with the processed camera image:

```
TfLiteStatus result = interpreter.Invoke();
```

The `Invoke()` function executes the neural-network operations and produces output values.

Espressif's example similarly captures image data, places it into the input tensor, calls `interpreter->Invoke()`, and then reads the output tensor to process the prediction.

Step 12: Interpret the prediction

Step 13: Take security action

Once the classification result is available, the ESP32 can control external devices through GPIO.

The basic logic could be:

```
if (predicted_class == EMPLOYEE &&
```

```
    confidence > 0.80) {
```

```
    // Grant access
```

```
    digitalWrite(GREEN_LED, HIGH);
```

```
    digitalWrite(DOOR_LOCK, HIGH);
```

```
}
```

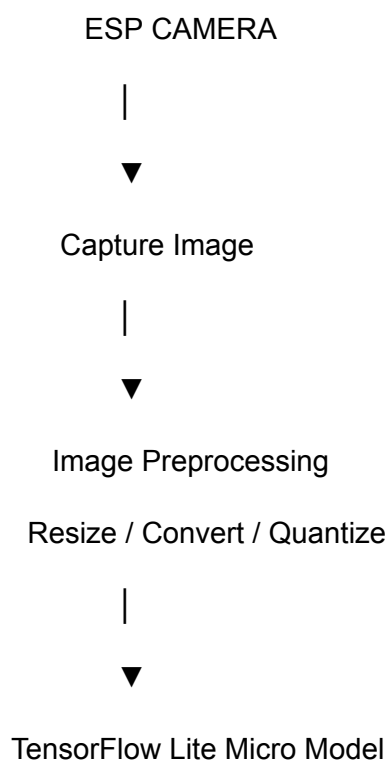
```
else {  
  
    // Deny access  
  
    digitalWrite(RED_LED, HIGH);  
  
    digitalWrite(BUZZER, HIGH);  
  
}
```

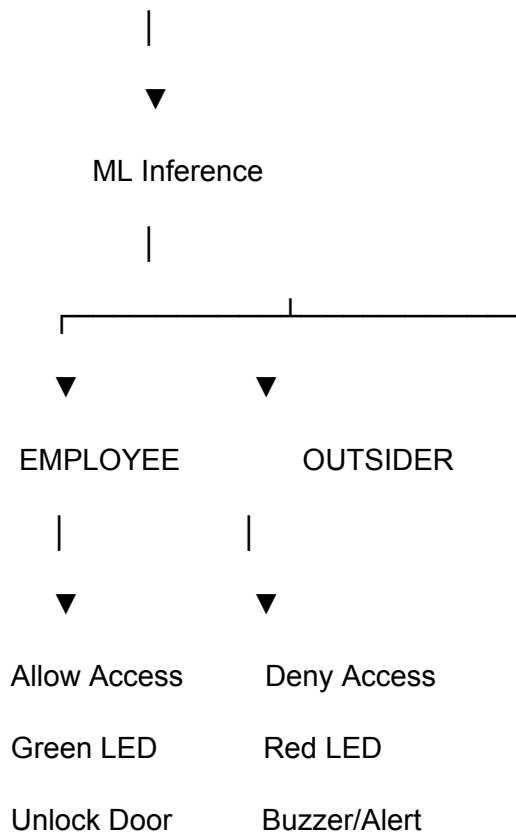
Step 14: Build and flash the application

After integrating:

- Camera code
- Custom TFLite model
- TensorFlow Lite Micro
- Image preprocessing
- Security output logic

Final working process of the proposed security system





Discussion:

This experiment extended the earlier Pi Camera setup into full video streaming and applied machine learning. Getting the Pi camera stream to display smoothly required understanding that picamera hands frames to OpenCV as raw NumPy arrays rather than as a ready-made video widget, and that the preview must be cleared (`truncate(0)`) after every frame to avoid a growing memory buffer. Streaming from the mobile phone was conceptually simpler but depended entirely on both devices being on the same Wi-Fi network; a couple of early attempts failed silently until the phone's IP Webcam URL was matched exactly. Researching the ESP32/TinyML deployment (Task 2) was the most time-consuming part, since it required piecing together the model-conversion, quantization, and C-array-embedding steps from several different sources rather than one single guide. Tracing through how `TFLite_detection_webcam.py` processes each frame also clarified why the pre-trained model and the detection code are downloaded from two different places — the script is reusable “plumbing”, while the model file is the actual learned intelligence, and either one can be swapped for another later on.

References

[1] United International University, Department of Computer Science & Engineering, *Experiment 04: Image and Video Processing with Raspberry Pi by Interfacing Pi Camera*, Course Lab Manual.

[2] TensorFlow, *TensorFlow Lite for Microcontrollers (TFLM)*, GitHub Repository.
[TensorFlow Lite Micro Repository](#)

[3] Espressif Systems, *TensorFlow Lite Micro for Espressif Chipsets*, GitHub Repository. This source provides ESP-IDF integration, examples, build instructions, flashing, monitoring, and ESP-NN optimization information.
[Espressif ESP-TFLite-Micro Repository](#)

[4] Espressif Systems, *ESP32 Camera Application / Camera Documentation*. This covers camera configuration, supported formats, resolution, and camera integration.
[Espressif Camera Documentation](#)

[5] Espressif Systems, *TensorFlow Lite Micro Person Detection – Camera Configuration*. This provides an example of configuring camera support for the TFLite Micro person-detection application.
[Person Detection Camera Configuration](#)

[6] TensorFlow Lite Micro, *New Platform Support / Integration Guide*. This explains integrating TFLite Micro with target hardware and peripherals such as cameras, as well as optimized kernel implementations.
[TFLite Micro Platform Support Guide](#)

[7] Espressif Systems, *TensorFlow Lite Micro Person Detection Example – main_functions.cc*. This demonstrates camera initialization, image acquisition, tensor input, `AllocateTensors()`, and model inference using `Invoke()`.
[Person Detection Example Code](#)

[8] Espressif Systems, *ESP-TFLite-Micro Person Detection Example*, ESP Component Registry. This describes deploying a small int8-quantized neural network for camera-based person detection on ESP32 devices.
[ESP-TFLite-Micro Person Detection Example](#)

